

# JavaScript Fetch API — Answers

GET, POST, Promise chaining, filtering, and DevTools Network inspection

## Task 1

Use `fetch()` to get posts from JSONPlaceholder and display the first 5 post titles in the browser console.

```
fetch("https://jsonplaceholder.typicode.com/posts")
  .then(response => response.json())
  .then(posts => {
    posts.slice(0, 5).forEach(post => {
      console.log(post.title);
    });
  })
  .catch(error => console.error("Error:", error));
```

Logs the titles of posts 1 through 5.

## Task 2

Repeat the `fetch()` request and display the first 5 post titles in the console.

```
fetch("https://jsonplaceholder.typicode.com/posts")
  .then(response => response.json())
  .then(posts => {
    posts.slice(0, 5).forEach(post => console.log(post.title));
  })
  .catch(error => console.error("Error:", error));
```

Logs the first five post titles.

## Task 3

GET the posts endpoint with `fetch()`, parse the JSON, and display the first 5 titles.

```
fetch("https://jsonplaceholder.typicode.com/posts")
  .then(response => response.json())
  .then(posts => {
    posts.slice(0, 5).forEach(post => {
      console.log(`Post: ${post.title}`);
    });
  })
  .catch(error => console.error(error));
```

Logs five post titles with a Post: prefix.

## Task 4

Build `fetchTopMovies()` using the albums endpoint, parse JSON, and log the first 3 titles as trending movies.

```
function fetchTopMovies() {
  fetch("https://jsonplaceholder.typicode.com/albums")
    .then(response => response.json())
    .then(albums => {
      console.log("Trending Movies:");
      albums.slice(0, 3).forEach(album => {
        console.log(album.title);
      });
    })
    .catch(error => console.error("Error:", error));
}
```

`fetchTopMovies();`

Logs the first three album titles as "Trending Movies."

## Task 5

Create the same `fetchTopMovies()` function and log the first three titles.

```
function fetchTopMovies() {
```

```

fetch("https://jsonplaceholder.typicode.com/albums")
  .then(response => response.json())
  .then(albums => {
    albums.slice(0, 3).forEach(movie => {
      console.log(`Trending: ${movie.title}`);
    });
  })
  .catch(error => console.error(error));
}

```

fetchTopMovies();

Logs the first three album titles as trending items.

## Task 6

Send a POST request with a JSON title/body, log the response object, and check whether id is present.

```

fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    title: "My New Post",
    body: "This is a sample post created with fetch()."
  })
})
  .then(response => response.json())
  .then(data => {
    console.log("Response:", data);

    if ("id" in data) {
      console.log("id property is present:", data.id);
    } else {
      console.log("id property is not present");
    }
  })
  .catch(error => console.error("Error:", error));

```

JSONPlaceholder normally returns a created object containing an id.

## Task 7

Create addNewPlaylist() that POSTs a playlist name and description and logs the response JSON.

```

function addNewPlaylist() {
  const playlist = {
    name: "My Chill Playlist",
    description: "Songs for relaxing and studying."
  };

  fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(playlist)
  })
    .then(response => response.json())
    .then(data => {
      console.log("Playlist response:", data);
    })
    .catch(error => console.error("Error:", error));
}

```

addNewPlaylist();

Logs the JSON response returned by JSONPlaceholder.

## Task 8

Create another addNewPlaylist() implementation using POST and JSON headers.

```

function addNewPlaylist() {
  const playlist = {
    name: "Bollywood Hits",
    description: "My favorite Bollywood songs."
  };
}

```

```

fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(playlist)
})
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error(error));
}

addNewPlaylist();

```

Logs the returned playlist/post object.

## Task 9

Fetch users, parse JSON, and display all user emails in one alert separated by commas.

```

fetch("https://jsonplaceholder.typicode.com/users")
.then(response => response.json())
.then(users => {
  const emails = users.map(user => user.email);
  alert(emails.join(", "));
})
.catch(error => console.error("Error:", error));

```

One alert box contains all user email addresses separated by commas.

## Task 10

Fetch user 2, use that user's id to fetch their posts, then log the user's name and post titles.

```

fetch("https://jsonplaceholder.typicode.com/users/2")
.then(response => response.json())
.then(user => {
  return fetch(
    `https://jsonplaceholder.typicode.com/posts?userId=${user.id}`
  )
  .then(response => response.json())
  .then(posts => {
    console.log("User:", user.name);
    console.log("Post titles:");

    posts.forEach(post => {
      console.log(post.title);
    });
  });
})
.catch(error => console.error("Error:", error));

```

First request gets the user; the second request uses user.id to get that user's posts.

## Task 11

Repeat the two-fetch chain: get user 2, fetch their posts by id, then log the name and titles.

```

fetch("https://jsonplaceholder.typicode.com/users/2")
.then(response => response.json())
.then(user => {
  return fetch(
    `https://jsonplaceholder.typicode.com/posts?userId=${user.id}`
  )
  .then(response => response.json())
  .then(posts => {
    console.log(`${user.name}'s posts:`);

    posts.forEach(post => {
      console.log(post.title);
    });
  });
})
.catch(error => console.error(error));

```

Logs user 2's name followed by the titles of that user's posts.

## Task 12

Use DevTools Network tab to make a GET request, inspect the request/response, and take a screenshot.

```
// Run this in the browser Console:
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => response.json())
  .then(users => console.log(users))
  .catch(error => console.error(error));
```

Steps:

1. Open the page in Chrome/Edge.
2. Press F12 (or Ctrl + Shift + I).
3. Open the Network tab.
4. Clear existing requests if needed.
5. Run the code above in the Console.
6. Return to Network and click the "users" request.
7. Inspect Headers, Payload (if shown), Preview/Response, and Status.
8. Take a screenshot showing the request and response details.

Two-line explanation:

The browser sends an HTTP GET request to the users endpoint and receives an HTTP response.

The response contains JSON user data, which response.json() parses into a JavaScript array.

## Task 13

Open DevTools Network, run the GET request, inspect the request/response cycle, and take a screenshot.

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => {
    console.log("Status:", response.status);
    return response.json();
  })
  .then(users => console.log("Users:", users));
```

Screenshot procedure:

Open DevTools → Network → run the fetch in Console → select the users request → inspect Request URL, Request Method, Status Code, Response/Preview, and Headers → take a screenshot.

What you should see:

The request method is GET and the response contains JSON representing the users. The response status should indicate a successful request when the service is reachable.

## Task 14

Open DevTools Network, run the GET request, inspect request and response details, and take a screenshot.

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => response.json())
  .then(data => console.log("Response data:", data))
  .catch(error => console.error("Request failed:", error));
```

Screenshot procedure:

1. Open Developer Tools.
2. Select Network.
3. Run the fetch code in Console.
4. Click the users request.
5. Capture the Request URL, GET method, status, response headers, and JSON response.

Description:

The Network panel shows the outgoing GET request and the server's response. The response body contains the user records as JSON, which fetch() can parse with response.json().

## Task 15

Fetch albums, filter only albums where userId is 2, and display their titles. Use only fetch(), .then(), and array methods.

```
fetch("https://jsonplaceholder.typicode.com/albums")
  .then(response => response.json())
  .then(albums => {
    const user2Albums = albums.filter(album => album.userId === 2);

    user2Albums.forEach(album => {
      console.log(album.title);
    });
  })
  .catch(error => console.error("Error:", error));
```

Only album titles belonging to userId 2 are logged. No async/await is used.